

Relatório de Atividade Prática

NÚMERO DO LABORATÓRIO #2

Integrantes do Grupo (EM ORDEM ALFABÉTICA)

Nome: Dandi Mateus Aguiar Reis	RA: 10444938
Nome: Leandro Solovjovas dos Santos	RA: 10438426
Nome: Pedro Henrique Marques Ferreira	RA: 10723681

EXERCÍCIO 1

PRINT DO CÓDIGO FONTE

```
1 # =====
2 # Laboratório 2 - Exercício 1
3 # Fatorial usando Função em Assembly MIPS
4 #
5 # Dandi Mateus Aguiar Reis          RA: 10444938
6 # Leandro Solovjovas dos Santos     RA: 10438426
7 # Pedro Henrique Marques Ferreira   RA: 10723681
8 # =====
9
10 .data
11
12 msg_prompt: .asciiz "Digite um numero inteiro positivo: "
13 msg_erro:   .asciiz "Entrada invalida! O numero deve ser maior que zero.\n\n"
14 msg_result: .asciiz "O fatorial e: "
15
```

```
16 .text
17 .globl main
18
19 #####
20 # Programa Principal
21 #####
22
23 main:
24
25 loop_leitura:
26
27     # Solicita um numero ao usuario
28     li $v0, 4
29     la $a0, msg_prompt
30     syscall
31
32     # Le o numero inteiro
33     li $v0, 5
34     syscall
35     move $t0, $v0
36
37     # Valida a entrada.
38     # Se o numero for menor ou igual a zero, solicita uma nova entrada.
39     blez $t0, entrada_invalida
40
41     # Passa o numero para a funcao fatorial utilizando o registrador $a0.
42     move $a0, $t0
43
44     # Chama a funcao fatorial
45     jal fatorial
46
47     # Salva o resultado retornado em $v0
48     move $t1, $v0
49
```

```

50     # Imprime a mensagem do resultado
51     li $v0, 4
52     la $a0, msg_result
53     syscall
54
55     # Imprime o resultado do fatorial
56     li $v0, 1
57     move $a0, $t1
58     syscall
59
60     # Encerra o programa
61     li $v0, 10
62     syscall
63
64
65     #####
66     # Tratamento de entrada invalida
67     #####
68
69     entrada_invalida:
70
71     # Exibe mensagem de erro
72     li $v0, 4
73     la $a0, msg_erro
74     syscall
75
76     # Solicita novamente a entrada
77     j loop_leitura
78
79

```

```

80     #####
81     # Funcao: fatorial
82     #
83     # Entrada:
84     #   $a0 = numero para calcular o fatorial
85     #
86     # Saida:
87     #   $v0 = resultado do fatorial
88     #####
89
90     fatorial:
91
92     # Inicializa o resultado com 1
93     li $v0, 1
94
95     # Copia o numero para o contador
96     move $t2, $a0
97
98     loop_fatorial:
99
100    # Verifica se o contador chegou a zero
101    blez $t2, fim_fatorial
102
103    # Multiplica o resultado pelo contador
104    mul $v0, $v0, $t2
105
106    # Decrementa o contador
107    subi $t2, $t2, 1
108
109    # Repete o processo
110    j loop_fatorial
111

```

```

112    fim_fatorial:
113
114    # Retorna para o programa principal
115    jr $ra
116

```

PRINT COM TESTES DE FUNCIONALIDADE

```
Digite um numero inteiro positivo: 4
O fatorial e: 24
-- program is finished running --

Digite um numero inteiro positivo: -1
Entrada invalida! O numero deve ser maior que zero.

Digite um numero inteiro positivo: 0
Entrada invalida! O numero deve ser maior que zero.

Digite um numero inteiro positivo: -44
Entrada invalida! O numero deve ser maior que zero.

Digite um numero inteiro positivo: 6
O fatorial e: 720
-- program is finished running --

Digite um numero inteiro positivo: 9
O fatorial e: 362880
-- program is finished running --
```

Clear

COMENTÁRIOS GERAIS

A solução foi desenvolvida utilizando uma função para calcular o fatorial do número informado pelo usuário. Primeiro, o programa solicita a entrada e verifica se o valor é maior que zero. Caso o valor seja inválido, uma mensagem de erro é apresentada e o programa solicita uma nova entrada. Depois da validação, o número é passado para a função fatorial através do registrador \$a0. A função utiliza um laço de repetição, multiplicando o resultado pelo contador até que ele chegue a zero. O resultado é retornado pelo registrador \$v0 e então apresentado na tela.

A principal dificuldade encontrada foi entender como realizar a chamada e o retorno de uma função em Assembly MIPS utilizando as instruções jal e jr, além de organizar os registradores utilizados durante o cálculo.

A solução atende ao que foi solicitado no exercício, pois realiza a leitura e validação da entrada, utiliza a função fatorial com jal e jr, calcula o resultado e imprime o fatorial na tela.

Como observação, o cálculo foi realizado de forma iterativa, sem utilizar pilha, utilizando apenas os registradores necessários para controlar o contador e o resultado.

EXERCÍCIO 2

PRINT DO CÓDIGO FONTE

```
1  # =====
2  # Laboratório 2 - Exercício 2
3  # Sequência de Fibonacci usando Função em Assembly MIPS
4  #
5  # Dandi Mateus Aguiar Reis          RA: 10444938
6  # Leandro Solovjovas dos Santos     RA: 10438426
7  # Pedro Henrique Marques Ferreira   RA: 10723681
8  # =====
9
10 .data
11
12 msg_prompt: .ascii "Digite a quantidade de termos da sequencia de Fibonacci: "
13 msg_erro:   .ascii "Entrada invalida! O numero deve ser maior que zero.\n\n"
14 msg_virgula: .ascii ", "
15
```

```
16 .text
17 .globl main
18
19 #####
20 # Programa Principal
21 #####
22
23 main:
24
25 loop_leitura:
26
27     # Solicita a quantidade de termos
28     li $v0, 4
29     la $a0, msg_prompt
30     syscall
31
32     # Le a quantidade de termos
33     li $v0, 5
34     syscall
35     move $s0, $v0
36
37     # Valida a entrada.
38     # Se for menor ou igual a zero, solicita novamente.
39     blez $s0, entrada_invalida
40
41     # Passa a quantidade de termos para a funcao
42     move $a0, $s0
43
44     # Chama a funcao Fibonacci
45     jal fibonacci
46
47     # Encerra o programa
48     li $v0, 10
49     syscall
50
```

```
51
52 #####
53 # Tratamento de entrada invalida
54 #####
55
56 entrada_invalida:
57
58     # Exibe mensagem de erro
59     li $v0, 4
60     la $a0, msg_erro
61     syscall
62
63     # Solicita novamente a entrada
64     j loop_leitura
65
66
```

```

67 #####
68 # Funcao: fibonacci
69 #
70 # Entrada:
71 #   $a0 = quantidade de termos
72 #
73 # Gera a sequencia:
74 #   0, 1, 1, 2, 3, 5, ...
75 #####
76
77 fibonacci:
78
79     # $t2 = quantidade de termos restantes
80     move $t2, $a0
81
82     # $t0 = F0
83     li $t0, 0
84
85     # $t1 = F1
86     li $t1, 1
87
88     # Imprime o primeiro termo
89     li $v0, 1
90     move $a0, $t0
91     syscall
92
93     # Um termo ja foi impresso
94     subi $t2, $t2, 1
95
96
97 loop_fibonacci:
98
99     # Verifica se todos os termos foram impressos
100    blez $t2, fim_fibonacci
101
102    # Imprime a virgula entre os termos
103    li $v0, 4
104    la $a0, msg_virgula
105    syscall
106
107    # Imprime o termo atual
108    li $v0, 1
109    move $a0, $t1
110    syscall
111
112    # Calcula o proximo termo
113    #  $F(n) = F(n-1) + F(n-2)$ 
114    add $t3, $t0, $t1
115
116    # Atualiza os termos
117    move $t0, $t1
118    move $t1, $t3
119
120    # Decrementa a quantidade restante
121    subi $t2, $t2, 1
122
123    # Repete o processo
124    j loop_fibonacci
125
126
127 fim_fibonacci:
128
129     # Retorna para o programa principal
130     jr $ra

```

PRINT COM TESTES DE FUNCIONALIDADE

Digite a quantidade de termos da sequencia de Fibonacci: 5

0, 1, 1, 2, 3

-- program is finished running --

Digite a quantidade de termos da sequencia de Fibonacci: -1

Entrada invalida! O numero deve ser maior que zero.

Digite a quantidade de termos da sequencia de Fibonacci: 0

Entrada invalida! O numero deve ser maior que zero.

Digite a quantidade de termos da sequencia de Fibonacci: 22

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946

-- program is finished running --

Digite a quantidade de termos da sequencia de Fibonacci: 6

0, 1, 1, 2, 3, 5

-- program is finished running --

Clear

COMENTÁRIOS GERAIS

A solução utiliza uma função chamada Fibonacci para gerar a quantidade de termos informada pelo usuário. Inicialmente, o programa solicita a quantidade de termos e verifica se o valor é maior que zero. Depois da validação, a quantidade é passada para a função através do registrador `%a0`. A função começa com os valores 0 e 1 e utiliza um laço de repetição para calcular os próximos termos somando os dois valores anteriores. Os termos são impressos na mesma linha e separados por vírgulas.

A principal dificuldade encontrada foi controlar a quantidade de termos que deveriam ser impressos e atualizar corretamente os dois valores anteriores da sequência a cada repetição. Também foi necessário cuidar da impressão das vírgulas para que a sequência ficasse no formato solicitado.

A solução atende ao que foi solicitado, pois realiza a leitura e validação da quantidade de termos, utiliza uma função chamada através de `jal` e retornada com `jr`, gera a sequência de Fibonacci e imprime os valores separados por vírgulas.

Como observação, a sequência é calculada de forma iterativa e não utiliza pilha. Os valores necessários para gerar os próximos termos são mantidos diretamente nos registradores.

EXERCÍCIO 3

CÓDIGO FONTE

```
1  # =====
2  # Laboratório 2 - Exercício 3
3  # Conversao Decimal para Binario em Assembly MIPS
4  #
5  # Dandi Mateus Aguiar Reis          RA: 10444938
6  # Leandro Solovjovas dos Santos    RA: 10438426
7  # Pedro Henrique Marques Ferreira  RA: 10723681
8  # =====
9
10 .data
11
12 # Espaco para armazenar os restos das divisoes
13 binario: .space 32
14
15 msg_prompt: .asciiz "Digite um numero decimal inteiro positivo: "
16 msg_erro: .asciiz "Entrada invalida! O numero deve ser maior que zero.\n\n"
17 msg_result: .asciiz "O numero em binario e: "
18
19 .text
20 .globl main
21
22 #####
23 # Programa Principal
24 #####
25
26 main:
27
28 loop_leitura:
29
30     # Solicita um numero decimal ao usuario
31     li $v0, 4
32     la $a0, msg_prompt
33     syscall
34
35     # Le o numero inteiro
36     li $v0, 5
37     syscall
38     move $s0, $v0
39
40     # Valida a entrada.
41     # Se for menor ou igual a zero, solicita novamente.
42     blez $s0, entrada_invalida
43
44     # Passa o numero para a funcao atraves do registrador $a0.
45     move $a0, $s0
46
47     # Chama a funcao de conversao
48     jal decimal_para_binario
49
50     # Encerra o programa
51     li $v0, 10
52     syscall
53
54
55 #####
56 # Tratamento de entrada invalida
57 #####
58
59 entrada_invalida:
60
61     # Exibe mensagem de erro
62     li $v0, 4
63     la $a0, msg_erro
64     syscall
65
66     # Solicita novamente a entrada
67     j loop_leitura
68
69
```

```

70 #####
71 # Funcao: decimal_para_binario
72 #
73 # Entrada:
74 #   $a0 = numero decimal positivo
75 #
76 # Funcionamento:
77 #   Divide sucessivamente o numero por 2.
78 #   Os restos sao armazenados na memoria.
79 #   Depois, os restos sao lidos de tras para frente.
80 #####
81
82 decimal_para_binario:
83
84     # $t0 = numero atual
85     move $t0, $a0
86
87     # $t1 = divisor 2
88     li $t1, 2
89
90     # $t2 = indice da memoria
91     li $t2, 0
92
93
94 loop_divisoes:
95
96     # Divide o numero por 2
97     divu $t0, $t1
98
99     # $t3 recebe o resto
100    mfhi $t3
101
102    # $t0 recebe o quociente
103    mflo $t0
104
105    # Armazena o resto na memoria
106    sb $t3, binario($t2)
107
108    # Avanca para a proxima posicao
109    addi $t2, $t2, 1
110
111    # Continua enquanto o quociente for maior que zero
112    bgtz $t0, loop_divisoes
113
114
115 #####
116 # Impressao do resultado
117 #####
118
119     # Imprime a mensagem do resultado
120     li $v0, 4
121     la $a0, msg_result
122     syscall
123
124
125 loop_imprime_binario:
126
127     # Os restos foram armazenados do menos significativo
128     # para o mais significativo.
129     # Por isso, a leitura e feita de tras para frente.
130
131     subi $t2, $t2, 1
132
133     # Verifica se todos os bits foram impressos
134     bltz $t2, fim_binario
135
136     # Carrega o bit armazenado
137     lb $a0, binario($t2)
138
139
140     # Imprime o bit
141     li $v0, 1
142     syscall
143
144     # Continua a impressao
145     j loop_imprime_binario
146
147 fim_binario:
148
149     # Retorna para o programa principal
150     jr $ra
151

```

TESTES DE FUNCIONALIDADE

Digite um numero decimal inteiro positivo: 0
Entrada invalida! O numero deve ser maior que zero.

Digite um numero decimal inteiro positivo: -23
Entrada invalida! O numero deve ser maior que zero.

Digite um numero decimal inteiro positivo: -1
Entrada invalida! O numero deve ser maior que zero.

Digite um numero decimal inteiro positivo: -88
Entrada invalida! O numero deve ser maior que zero.

Digite um numero decimal inteiro positivo: 1
O numero em binario e: 1
-- program is finished running --

Digite um numero decimal inteiro positivo: 22
O numero em binario e: 10110
-- program is finished running --

Clear

Digite um numero decimal inteiro positivo: 100
O numero em binario e: 1100100
-- program is finished running --

Digite um numero decimal inteiro positivo: 90
O numero em binario e: 1011010
-- program is finished running --

COMENTÁRIOS GERAIS

A solução realiza a conversão de um número decimal para binário utilizando o método das divisões sucessivas. Primeiro, o programa solicita um número inteiro positivo e verifica se a entrada é válida. Em seguida, o número é passado para a função `decimal_para_binario` através do registrador `%a0`. Dentro da função, o número é dividido sucessivamente por 2. A cada divisão, o resto, que corresponde a um bit da representação binária, é armazenado em uma área de memória reservada com `.space 32`. Como os restos são obtidos do bit menos significativo para o mais significativo, após o término das divisões eles são lidos de trás para frente e impressos na tela.

A principal dificuldade encontrada foi entender que os restos das divisões precisam ser armazenados na ordem em que são obtidos e depois percorridos no sentido inverso para que o número binário seja apresentado corretamente.

A solução atende ao que foi solicitado, pois realiza a leitura e validação do número, utiliza a função `decimal_para_binario` através de `jal` e `jr`, realiza a conversão pelo método das divisões sucessivas e imprime somente os bits necessários para representar o número.

Como observação, foi utilizado um espaço de memória de 32 bytes para armazenar os restos das divisões. Essa memória não corresponde a uma pilha; ela foi utilizada apenas para guardar temporariamente os bits durante a conversão.

OBSERVAÇÕES FINAIS

- **Gerar um PDF** deste arquivo antes de submeter na tarefa do Moodle;
- Além deste relatório, **enviar o arquivo .asm** do(s) programa(s) criado(s): Ex1.asm, Ex2.asm ...
- Cuidado com o tamanho e qualidade das imagens. Observe se é possível visualizar seu conteúdo!
- **ATENÇÃO:** Na tarefa do Moodle, apenas 1 (um) aluno da equipe deve submeter a atividade.
- **O que será avaliado?**
 - Completude, clareza e organização do relatório. É obrigatório usar este *template*;
 - Funcionamento correto dos programas, atendendo ao que foi solicitado. O quão fiel cada programa é quanto à descrição do enunciado;
 - Completude dos testes realizados;
 - Indentação, comentários, organização e legibilidade dos códigos;